

Software Requirements Specification (SRS)

Project Title: File Compression Tool (Basic ZIP Implementation) **Document Version:** 1.0 **Date:** September 28, 2026 **Standard:** IEEE Std 830-1998 **Team Designation:** Team-E17 **Target Language:** C / C++

Team Members & Details

- **Mahesh Kumar B** — SRN: PES1UG24CS255
 - **Nishanth T Chigilar** — SRN: PES1UG24CS302
 - **Mudit Chaturvedi** — SRN: PES1UG24CS279
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) specifies the functional, non-functional, interface, and security requirements for the **File Compression Tool (Basic ZIP Implementation)**. The software is a command-line utility built in C/C++ that utilizes Huffman coding for lossless data compression and decompression. This document serves as the formal baseline for development, validation, and testing across the Software Development Life Cycle (SDLC).

1.2 Document Conventions

- **FR-xx**: Functional Requirement
- **NFR-xx**: Non-Functional Requirement
- **SEC-OBJ-xx**: Security Objective
- **SEC-REQ-xx**: Security Requirement
- Priority levels: **High** (Mandatory for core delivery), **Medium** (Required for complete functionality), **Low** (Optional enhancement).

1.3 Intended Audience

This document is intended for project developers (Team-E17), software evaluators, and system testers.

1.4 Project Scope

The File Compression Tool is a standalone CLI program written in C/C++. It reads raw binary/text files, calculates character frequency distributions, constructs an optimal Huffman tree, serializes the tree and encoded bitstream into a custom archive format (.czip), and reconstructs the original file losslessly.

1.5 Definitions, Acronyms, and Abbreviations

- **CLI**: Command Line Interface
- **Huffman Coding**: An entropy encoding algorithm used for lossless data compression.
- **Min-Heap / Priority Queue**: Data structure used to build the Huffman tree efficiently.

- **Header:** Metadata prepended to the encoded file containing original file info and tree deserialization mapping.

1.6 References

- IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
 - ISO/IEC 14882:2020: *Programming Languages — C++*.
-

2. Overall Description

2.1 Product Perspective

The tool is a self-contained executable operating within the terminal environment (Linux/Unix POSIX, Windows CMD/PowerShell). It interacts directly with the local OS file system for reading and writing files.

2.2 Product Functions

1. **Compress File:** Reads input file, constructs Huffman codes, writes compressed payload with metadata header.
2. **Decompress File:** Reads `.czip` archive, parses metadata header, reconstructs Huffman tree, decodes payload to restore exact original file.
3. **Display Stats:** Outputs original size, compressed size, compression ratio percentage, and processing execution time.
4. **Archive Integrity Check:** Validates header magic bytes and payload bitstream consistency.

2.3 User Characteristics

Users are assumed to be familiar with standard command-line interaction, path parameters, and basic file operations.

2.4 Design and Implementation Constraints

1. **Language Constraint:** Must be written purely in standard C/C++ without external third-party compression libraries.
2. **Memory Constraint:** Algorithm must execute within $O(N)$ auxiliary memory relative to the size of the unique alphabet (max 256 unique byte values).
3. **Platform Independence:** Source code must compile cleanly using `GCC`, `Clang`, or `MSVC`.

2.5 Assumptions and Dependencies

- File system provides sufficient read/write permissions for specified input and output paths.
 - Available disk storage is at least equal to the input file size during decompression operations.
-

3. Specific Requirements

3.1 Functional Requirements

Requirement ID	Description	Priority	Measurable Acceptance Criteria
FR-01	File Compression	High	Execution of <code>czip -c <input></code> creates an output file <code>.czip</code> containing encoded bitstream and header.
FR-02	File Decompression	High	Execution of <code>czip -d <archive.czip></code> reproduces original file with 100% byte identity (<code>cmp</code> clean).
FR-03	Frequency Table Analysis	High	Reads file byte-by-byte (values <code>0x00–0xFF</code>) and populates a 256-element frequency map accurately.
FR-04	Huffman Tree Construction	High	Builds a deterministic binary tree using a Min-Heap priority queue based on byte frequencies.
FR-05	Header Serialization	High	Writes magic byte signature (<code>0x435A</code> / "CZ"), original filename length, original file byte size, and canonical frequency table.
FR-06	Bitstream Packing	High	Packs variable-length Huffman codes into standard 8-bit byte buffers with bit-shifting operations.
FR-07	CLI Status Output	Medium	Displays original size (bytes), compressed size (bytes), compression percentage savings, and execution time (ms).
FR-08	Help & Usage Menu	Low	Invoking <code>czip --help</code> or invalid syntax prints usage syntax and available flag parameters.

3.2 Non-Functional Requirements

Performance Requirements

- **NFR-01 (Throughput):** Compression throughput shall exceed **10 MB/s** on a standard dual-core x86-64 CPU for general text and uncompressed binary files.
- **NFR-02 (Memory Overhead):** Maximum peak heap RAM usage shall not exceed **50 MB** for input file sizes up to 500 MB.

Reliability & Accuracy

- **NFR-03 (Zero Data Loss):** Lossless guarantee. The decompressed output file must have a cryptographic SHA-256 hash strictly identical to the original pre-compressed file.

Usability & Error Handling

- **NFR-04 (CLI Feedback):** Clear error messages must be printed to `stderr` upon failure.
- **NFR-05 (Exit Codes):** The process shall return standard OS exit codes (`0` for success, `1` for execution failure, `2` for invalid user input).

4. Security Requirements

4.1 Security Objectives

- **SEC-OBJ-01 (Data Integrity Assurance):** Ensure compressed archives cannot be modified, corrupted, or truncated without immediate detection during decompression.
- **SEC-OBJ-02 (Memory Safety & System Resiliency):** Protect the host execution environment against malicious or corrupted archive files designed to trigger buffer overflow vulnerabilities.

4.2 Security Requirements

- **SEC-REQ-01 (Header Magic Byte & Header Bounds Validation):**
 - The application **shall verify** the 2-byte magic header (**0x435A**) and parse serialized file size metadata prior to memory allocation.
 - If the magic byte signature is invalid, the application **must abort** immediately with error code **SEC_ERR_BAD_HEADER**.
 - **SEC-REQ-02 (Safe Buffer & Out-of-Bounds Memory Handling):**
 - Bitstream reading and tree traversal during decompression **shall perform strict bounds checks** on array indices and dynamic buffers.
 - Malformed bitstreams that attempt to traverse non-existent child nodes **must fail gracefully** with exit code **SEC_ERR_MALFORMED_STREAM**.
-

5. UML Use Case Specification

5.1 Primary Actors

- **User / Student Evaluator:** Interacts with the executable via terminal interface.
- **Host Operating System / File System:** Provides file access, read/write I/O handles.

5.2 Use Case List

1. **UC-01: Compress Target File**
2. **UC-02: Decompress CZIP Archive**
3. **UC-03: Display Compression Statistics**
4. **UC-04: Validate Archive Integrity**